Sprint 10 · Reto «¿Vale la pena la red?»

Ficha completa del reto, para consultar. Para trabajar, el reto en una página. Qué se sube y cómo se evalúa: entrega y rúbrica.

- Volver al README del sprint.
- Si es vuestro primer reto: Cómo se trabaja un reto.

RETO **Ficha del reto**

1 Qué os pide el cliente

Qué os pide el cliente. La dirección de TurisData oye hablar de **redes neuronales** en todas partes y pregunta si debe **sustituir su modelo de cancelaciones por una red**. No quiere una opinión: quiere **evidencias, con el coste incluido**, para decidir si merece la pena el cambio. La pregunta del reto es *¿una red neuronal mejora al aprendizaje automático clásico, y a qué precio?*, y la respuesta puede ser **«no»**: un «no compensa» bien demostrado es tan válido y útil como un «sí». Vais a entrenar una red sencilla (un MLP, una red de capas de neuronas artificiales), a compararla con un modelo clásico de árboles (el *boosting*) y a escribir un **informe de 1–2 páginas** que termine en «Recomendamos… porque…».

Cómo se trabaja. No hace falta experiencia previa: os damos un **cuaderno de Colab casi hecho** (`S10_04`, con cuatro ejercicios con hueco y comprobaciones automáticas que os dicen si lo que escribís es correcto) y vuestro trabajo es completarlo, probar variantes, mirar las curvas, comparar con honestidad (con varias semillas, para que un número más alto no os engañe) y contarlo en el informe. En clase tenéis 10 minutos de planning (día 1) y unos **90 minutos de reto + 40 de entrega, review y retro** (día 2). Los seis pasos suman 145 minutos de trabajo: lo que no quepa en clase va a casa o a la 6.ª hora. El equipo decide hasta dónde llega: mínimo, estándar o extra.

2 · Recursos del reto

Recurso	Para qué lo usaréis	Dónde está
El cuaderno del reto `S10_04_reto_vale_la_pena`	Es el esqueleto: partición, boosting de referencia, MLP con 3 semillas, curvas, comparación con intervalo y la variante de precio	Colab (basta con CPU; no hace falta GPU). Su código explicado lo cuenta celda a celda
Cuadernos de práctica `S10_02` y `S10_03`	Las pistas: cómo se construye un MLP en Keras y cómo se ven y se combaten el sobreajuste, el *dropout*, la L2 y la parada temprana	Colab; su código explicado
Presentaciones y apuntes	Repasar la red por dentro y cómo entrenarla: red desde cero, redes densas con Keras, entrenamiento y regularización (la diapositiva 11 es de este reto) y los apuntes de redes neuronales	Carpeta `1_Teoria`
Los datos `reservas_turisdata.csv`	6000 reservas **sintéticas** (inventadas), con la columna `cancelada` y el `precio_noche`	El cuaderno los descarga solo en su primera celda; no hace falta tocarlos
Google Colab (Python, Keras y scikit-learn ya instalados)	Ejecutar el cuaderno sin instalar nada	colab.research.google.com; al terminar, *Archivo → Descargar → Descargar .ipynb*
La plantilla del informe `plantilla_informe_comparativo.md`	Las seis partes del informe, con huecos	En GitHub: copiad el texto con el botón *Raw*
Cómo entregar en GitHub	Subir el cuaderno, el informe y el README a `sprint-10/` desde el navegador	Guía de entrega
Autoevaluación y coevaluación	Cada persona rellena su hoja	Hoja del sprint
Un asistente de IA (opcional)	Entender conceptos, curvas y errores de Keras	Con las reglas de la «Ayuda 1» del paso a paso

3 · Guía paso a paso

Son seis pasos, de **145 minutos en total**. Cada uno dice **qué hacéis** (la lista corta), **cómo se hace** con el detalle, **qué deberíais ver** y **cuándo está hecho**. Las ayudas (IA, errores, atascos) están al final. En el horario del día 2 caben unos 105 minutos (90 de reto y 15 para poneros al día); los 40 que faltan se hacen en la **consolidación** (13:00–13:30) y en la **6.ª hora** (hasta las 14:30), y la subida puede hacerse hasta esa hora. Si el tiempo aprieta, completad el nivel mínimo y dejad lo demás para casa. Trabajad siempre en **una copia** del cuaderno que sea vuestra, sin tocar el original del curso.

Paso 1 · Elegir el problema y reproducir la referencia (día 2, 15 minutos)

1. Elegir **un problema**: cancelaciones (`cancelada`) o precio (`precio_noche`).
2. Abrir `S10_04` en Colab, hacer una copia y ejecutarlo celda a celda hasta la partición.
3. Hacer el ejercicio 1 (`evaluar`) y reproducir el modelo de referencia, anotando su métrica y su tiempo.
4. Fijar la partición entrenamiento / validación / prueba y no cambiarla nunca más.

En sencillo. TurisData pregunta si una red neuronal es mejor que lo que ya tiene. Para contestar con justicia hay que comparar los dos modelos **con las mismas reglas**, igual que dos candidatos que hacen el mismo examen. En este paso preparáis ese examen: repartís las 6000 reservas en tres montones (uno para estudiar, otro para ensayar y otro para el examen final), escribís la función que pone la nota (`evaluar`) y entrenáis al candidato veterano, un modelo de árboles llamado *boosting*, para tener una nota con la que comparar. La red llegará en el paso siguiente. No hace falta saber programar: casi todo el código ya está escrito y lo único que escribís es una función corta, que tenéis aquí línea a línea. Tampoco hace falta entender cómo funciona por dentro el boosting.

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
Celda	Cada caja del cuaderno. Las de texto se leen; las de código se **ejecutan** con el botón ▶ (o con `Mayús + Intro`).
Variable	Un nombre al que se guarda un valor: `y = …` guarda lo que haya a la derecha con el nombre `y`.
Función	Una receta con nombre: se escribe una vez con `def` y se usa las veces que haga falta. `evaluar(…)` será la receta que pone la nota a un modelo.
Diccionario	Una lista de parejas «nombre: valor» entre llaves, por ejemplo `{"AUC": 0.75, "F1": 0.5}`.
Partición	El reparto de las reservas en tres montones: con el de **entrenamiento** aprende el modelo; con el de **validación** se ensaya y se decide; el de **prueba** es el examen final y se mira **una sola vez**.
Semilla	Un número que fija el azar para que, al repetir el reparto, salga siempre lo mismo. Aquí, `SEMILLA = 42`.
AUC	Una nota de 0,5 a 1 que mide si el modelo pone las probabilidades más altas a las reservas que de verdad se cancelan. 0,5 es como tirar una moneda; 1 sería perfecto.
Boosting	Un modelo de «árboles» de decisión que se corrigen unos a otros. Aquí es la **referencia**: lo que ya funciona y contra lo que se compara la red.
`assert`	Una línea que comprueba algo y, si no se cumple, para con un `AssertionError` y un mensaje. Es el **punto de control** de cada ejercicio.

Cómo se hace

1. **Elegir el problema.** Elegid **cancelaciones**: es el que el cuaderno trae montado (¿la reserva termina con `cancelada` = 1 o con 0?) y los ejercicios 1 a 4 están pensados para él. El **precio** (`precio_noche`) es otro problema, predecir un número en euros, y queda como ampliación opcional para el final, con la sección 7 del cuaderno. Apuntad en el documento del equipo: «Problema: cancelaciones».
2. **Abrir vuestra copia del cuaderno.** Entrad en el cuaderno `S10_04` (se abre en Colab). Arriba, *Archivo → Guardar una copia en Drive*: así trabajáis en una copia vuestra y el original no se toca. Pulsad *Compartir* (arriba a la derecha) y añadid los correos del equipo con permiso de **Editor**. Una persona escribe y las demás miran y opinan; cambiad de turno cada pocos minutos.

3. **Ejecutar las celdas por orden, hasta la partición.** Pulsad ▶ en la celda «Preparación» (**siempre la primera**: crea la función `dato(…)`, que descarga el archivo de datos), después en la de importaciones y semilla (tarda unos segundos porque trae las librerías de redes neuronales) y después en «1. Datos y partición». Sabréis que han terminado porque aparece un y un número entre corchetes. **Ejecutad celda a celda, sin «Ejecutar todo»**: el cuaderno tiene huecos (`...`) que dan error hasta que los rellenéis, y más adelante hay una celda que usa la prueba y no debe ejecutarse antes de tiempo.

   La celda de la partición ya está escrita. No la modifiquéis; lo importante son estas tres líneas:

   ```
   X, y = df[num + cat], df["cancelada"].to_numpy()
   X_res, X_pru, y_res, y_pru = train_test_split(X, y, test_size=0.2, random_state=SEMILLA, stratify=y)
   X_ent, X_val, y_ent, y_val = train_test_split(X_res, y_res, test_size=0.2, random_state=SEMILLA, stratify=y_res)
   ```

   Qué hace cada línea:

   - `X, y = df[num + cat], df["cancelada"].to_numpy()`: X guarda las columnas con los datos de cada reserva (las de las listas `num` y `cat`) e y, la respuesta que se quiere adivinar: `cancelada` (1 = se canceló, 0 = no).
   - `X_res, X_pru, y_res, y_pru = train_test_split(…, test_size=0.2, …)`: aparta el 20 % de las reservas como **prueba** (`X_pru`, `y_pru`). Lo que queda se llama `_res` («resto»).
   - `X_ent, X_val, y_ent, y_val = train_test_split(X_res, …, test_size=0.2, …)`: del resto, el 20 % pasa a **validación** y el 80 % a **entrenamiento**. Salen un 64 % (0,8 × 0,8), un 16 % y un 20 %.
   - `random_state=SEMILLA`: hace que el reparto sea siempre el mismo. **No lo cambiéis**: si cambia la partición, los modelos ya no se pueden comparar entre sí.
   - `stratify=y`: reparte de forma que en cada montón quede la misma proporción de reservas canceladas (aproximadamente un 46 % en los tres).

   Las líneas siguientes de la celda (las de `pre`, `A_ent`, `A_val` y `A_pru`) convierten los datos en números listos para la red; las veréis en el paso 2.

4. **Ejercicio 1: la función `evaluar`.** Bajad hasta «2 · Una función para evaluar a todos por igual». La celda tiene una línea `# TODO` (una pista; Python la ignora), tres puntos `...` y varias líneas con `assert`. Borrad **solo los tres puntos** y escribid esto, respetando mayúsculas, comillas, paréntesis y la sangría de 4 espacios:

   ```
   def evaluar(y_real, prob, umbral=0.5):
       clases = (np.asarray(prob) >= umbral).astype(int)
       return {"AUC": roc_auc_score(y_real, prob),
               "AP": average_precision_score(y_real, prob),
               "F1": f1_score(y_real, clases),
               "exactitud": accuracy_score(y_real, clases)}
   ```

   Qué hace cada línea:

   - `def evaluar(y_real, prob, umbral=0.5):`: define la función. Recibe `y_real` (lo que pasó de verdad, 0 o 1), `prob` (la probabilidad de cancelar que da el modelo) y `umbral` (el corte; si no se dice nada, 0,5). **Los dos puntos del final y la sangría de las líneas de debajo son obligatorios.**
   - `clases = (np.asarray(prob) >= umbral).astype(int)`: convierte probabilidades en decisiones: 1 («cancelará») si la probabilidad llega al umbral y 0 si no. `np.asarray` se asegura de que `prob` sea una tabla de NumPy y `.astype(int)` cambia `True`/`False` por 1/0.
   - `return {…}`: devuelve un diccionario con las cuatro notas. **Los nombres de las claves no se pueden cambiar** (`"AUC"`, `"AP"`, `"F1"` y `"exactitud"`): el `assert` los comprueba.
   - `roc_auc_score(y_real, prob)` y `average_precision_score(y_real, prob)`: el AUC y el AP (otra nota basada en probabilidades). Se calculan con **`prob`**, no con las clases.
   - `f1_score(y_real, clases)` y `accuracy_score(y_real, clases)`: el F1 (equilibra no dejar escapar cancelaciones y no dar falsas alarmas) y la exactitud (aciertos entre total). Se calculan con **`clases`**, las decisiones.

   Pulsad ▶. Si no aparece nada, está bien: los `assert` solo hablan cuando algo falla.

5. **Entrenar la referencia (boosting).** Ejecutad la celda «3 · La referencia: boosting de árboles», que ya está escrita. Estas son sus líneas:

```
t0 = time.time()
boost = HistGradientBoostingClassifier(max_iter=200, learning_rate=0.05, max_depth=4, random_state=SEMILLA)
boost.fit(A_ent, y_ent)
t_boost = time.time() - t0
prob_boost_val, prob_boost_pru = boost.predict_proba(A_val)[:, 1], boost.predict_proba(A_pru)[:, 1]
print(f"Boosting entrenado en {t_boost:.1f} s · AUC validación = {roc_auc_score(y_val, prob_boost_val):.3f}")
```

   Qué hace cada línea:

   - `t0 = time.time()` y `t_boost = time.time() - t0`: miden cuántos segundos tarda en entrenar. **El tiempo forma parte del coste** que le contaréis al cliente.
   - `boost = HistGradientBoostingClassifier(…)`: crea el modelo (200 rondas de árboles, de profundidad máxima 4). `random_state=SEMILLA` fija su azar.
   - `boost.fit(A_ent, y_ent)`: **entrena**: aprende con las reservas de entrenamiento y sus respuestas.
   - `boost.predict_proba(A_val)[:, 1]`: pide la probabilidad de cancelar de cada reserva (`[:, 1]` es la columna del «sí cancela»). Se hace para validación y para prueba, pero **ahora solo se mira la de validación**: la de prueba se guarda para el veredicto del paso 5.
   - `print(f"…")`: escribe el tiempo y el AUC de validación con tres decimales.

6. **Anotar la referencia.** En el documento del equipo, apuntad el AUC de validación y el tiempo del boosting. **No se cambia la referencia para que la red gane.** Si vuestro equipo tiene un campeón del Sprint 8 (XGBoost, LightGBM, Random Forest…) y sabéis ponerlo en lugar del boosting, podéis usarlo; si no, vale el del cuaderno.

Qué deberíais ver.

- **La partición:** `(3840, 35) (960, 35) (1200, 35)`. Suman 6000 reservas y cada una son 35 números (9 de las columnas numéricas y 26 de las categorías convertidas en 0 y 1).
- **El ejercicio 1:** la celda no escribe nada. Si algo falla, aparece un `AssertionError` con un mensaje.
- **El boosting:** una línea como `Boosting entrenado en 0.3 s · AUC validación = 0.734`. El tiempo cambia según el ordenador (serán décimas de segundo); el AUC debe salir ≈ 0,73. Un AUC de 0,5 sería lo mismo que tirar una moneda; uno cercano a 1,0 haría sospechar de una fuga de datos (que el modelo hubiera visto la respuesta).

Hecho cuando: el cuaderno llega hasta el boosting sin errores, el ejercicio 1 pasa su comprobación y tenéis anotadas la métrica y el tiempo de la referencia.

Si algo falla

Veis…	Qué significa	Qué hacer
`NameError: name 'dato' is not defined` (o `'np'`, `'train_test_split'`, `'keras'`)	No se ha ejecutado una celda anterior, o Colab se desconectó y se olvidó de todo.	Ejecutad desde la primera celda hacia abajo, una a una, hasta la que falla.
`NameError: name 'evaluar' is not defined`	La celda del ejercicio 1 todavía tiene los `...` y no define la función.	Borrad los tres puntos, escribid la función y volved a pulsar ▶.
`TypeError: 'NoneType' object is not iterable`	La función no devuelve nada: falta el `return`.	Poned `return {…}` como última línea de la función, con la misma sangría.
`IndentationError: expected an indented block…`	Las líneas de dentro de la función no tienen sangría.	Dejad 4 espacios delante de las tres líneas de debajo de `def`.
`AssertionError: El diccionario debe tener las claves AUC, AP, F1 y exactitud`	Falta una clave o está mal escrita (por ejemplo, `precisión` en lugar de `exactitud`).	Copiad los cuatro nombres tal cual: `"AUC"`, `"AP"`, `"F1"`, `"exactitud"`.
`AssertionError: Revisa: AUC esperado 0.75 y exactitud 0.5 en la prueba mínima`	El AUC se ha calculado con las clases (0 y 1) en vez de con las probabilidades, o el umbral no se aplica bien.	AUC y AP con `prob`; F1 y exactitud con `clases`.
`ValueError: Classification metrics can't handle a mix of binary and continuous targets`	Se ha pasado `prob` (números decimales) a `f1_score` o a `accuracy_score`.	Pasadles `clases`.

Veis…	Qué significa	Qué hacer
El AUC de validación sale casi 1,000	Sospecha de fuga: el modelo ha visto la respuesta.	Comprobad que no habéis tocado las listas `num` y `cat` (la respuesta, `cancelada`, no puede ser una entrada).

Paso 2 · Primera red (día 2, 25 minutos)

1. Comprobar que los datos están escalados **ajustando el escalador solo con el entrenamiento**.
2. Entrenar un MLP pequeño en CPU, con un máximo de 40 épocas.
3. Dibujar la curva de aprendizaje: error en entrenamiento y en validación.
4. Hacer el ejercicio 2 (el AUC de cada semilla).

En sencillo. Una red neuronal es como un equipo de «calculadoras» pequeñas colocadas en filas: cada una hace una cuenta sencilla con los datos de la reserva y pasa su resultado a las de la fila siguiente. La última da un número entre 0 y 1: la probabilidad de que la reserva se cancele. Al principio las cuentas son casi al azar; **entrenar** es repasar las reservas una y otra vez, comprobar cuánto se equivoca y retocar un poquito las cuentas para que se equivoque menos. Cada repaso completo se llama **época**. Y como en un examen, hay dos notas: la de los ejercicios que ya ha visto (entrenamiento) y la de un simulacro con preguntas nuevas (validación). Aquí solo ejecutáis código que ya está escrito y lo leéis; escribís una única línea propia (ejercicio 2). **No hace falta** entender las matemáticas de dentro: basta con saber qué significa cada gráfica.

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
MLP	«Perceptrón multicapa»: la red más sencilla, formada por capas de neuronas, cada una conectada con todas las de la siguiente.
Capa y neurona	Una neurona es una cuenta pequeña (multiplicar, sumar y aplicar una regla). Una capa es un grupo de neuronas en paralelo. Aquí hay dos capas «ocultas» (de 64 y de 32 neuronas) y una de salida.
Época	Un repaso completo a todas las reservas de entrenamiento.
Pérdida (*loss*)	La nota del error: cuanto más baja, menos se equivoca la red.
Curva de aprendizaje	El dibujo de la pérdida época a época, una línea para entrenamiento y otra para validación.
Escalar	Poner todas las columnas numéricas en una escala parecida (media 0 y desviación 1) para que ninguna «grite» más que las otras. Las redes lo necesitan.
Fuga de datos	Que el modelo (o el escalador) se entere de algo del examen final antes de tiempo, y la nota salga mejor de lo que merece.
Parada temprana (*EarlyStopping*)	Parar el entrenamiento cuando la validación deja de mejorar y quedarse con la mejor versión.
Regularización (L2 y *dropout*)	Trucos para que la red no memorice: L2 castiga los pesos muy grandes; *dropout* apaga neuronas al azar mientras entrena.

Cómo se hace

1. **Entender cómo se escalaron los datos.** En el paso 1 ejecutasteis la celda de la sección 1, que ya escala los datos. Estas son las líneas que lo hacen (no las escribís):

```
pre = ColumnTransformer([("n", StandardScaler(), num), ("c", OneHotEncoder(handle_unknown="ignore"), cat)])
A_ent = pre.fit_transform(X_ent).astype("float32")
A_val = pre.transform(X_val).astype("float32")
A_pru = pre.transform(X_pru).astype("float32")
```

   Qué hace cada línea:

   - `pre = ColumnTransformer([…])`: define el preparador: las columnas numéricas (`num`) se escalan con `StandardScaler()` y las de texto (`cat`) se convierten en columnas de 0 y 1 con `OneHotEncoder` (una columna por cada valor posible).
   - `pre.fit_transform(X_ent)`: **`fit`** = «aprende» las medias y desviaciones, y **`transform`** = «aplica». Con `fit_transform` se hacen las dos cosas **solo con las reservas de entrenamiento**.
   - `pre.transform(X_val)` y `pre.transform(X_pru)`: aplican **las mismas medias y desviaciones aprendidas** a validación y prueba, sin volver a aprender nada. Si el escalador aprendiera también con la prueba, la red «se enteraría» del examen: eso es una **fuga**.
   - `.astype("float32")`: guarda los números con menos decimales, que es lo que espera la red.

   Para comprobarlo, añadid una celda nueva (botón **+ Código**) y ejecutad:

```
print(A_ent[:, :9].mean(axis=0).round(2))
print(A_val[:, :9].mean(axis=0).round(2))
```

   `A_ent[:, :9]` coge todas las filas y las 9 primeras columnas (las numéricas, que el preparador coloca delante); `.mean(axis=0)` calcula la media de cada columna y `.round(2)` deja dos decimales.

2. **Entrenar el MLP.** Ejecutad la celda de la sección 4, «El MLP, con las buenas prácticas del Sprint 10». Ya está escrita: **no cambiéis ningún número todavía** (en el paso 3 probaréis variantes). Tarda como mucho unos minutos y no escribe nada hasta terminar cada semilla. Este es el código:

```
def construir_mlp(n_entradas, capas=(64, 32), l2=0.003, dropout=0.3, tasa=1e-3):
    m = keras.Sequential([keras.layers.Input(shape=(n_entradas,))])
    for u in capas:
        m.add(keras.layers.Dense(u, activation="relu", kernel_regularizer=keras.regularizers.l2(l2)))
        m.add(keras.layers.Dropout(dropout))
    m.add(keras.layers.Dense(1, activation="sigmoid"))
    m.compile(optimizer=keras.optimizers.Adam(tasa), loss="binary_crossentropy", metrics=[keras.metrics.AUC(name="auc")])
    return m

modelos_mlp, historiales, tiempos = [], [], []
for semilla in (1, 2, 3):
    keras.utils.set_random_seed(semilla)
    t0 = time.time()
    m = construir_mlp(A_ent.shape[1])
    h = m.fit(A_ent, y_ent, validation_data=(A_val, y_val), epochs=40, batch_size=64, verbose=0,
              callbacks=[keras.callbacks.EarlyStopping(monitor="val_loss", patience=6, restore_best_weights=True)])
    tiempos.append(time.time() - t0)
    modelos_mlp.append(m); historiales.append(h)
    print(f"semilla {semilla}: {len(h.history['loss'])} épocas, AUC validación = {max(h.history['val_auc']):.3f} (máx.)")
```

Qué hace cada parte:

- `def construir_mlp(…)`: la receta que construye la red. Lo que va tras cada = es el ajuste por defecto: `capas=(64, 32)` (dos capas ocultas, de 64 y 32 neuronas), `l2=0.003`, `dropout=0.3` (apaga el 30 % de las neuronas) y `tasa=1e-3` (0,001: el tamaño de cada retoque, la **tasa de aprendizaje**).
- `keras.Sequential([keras.layers.Input(…)])`: una red vacía que recibe `n_entradas` números por reserva (aquí, 35).
- `for u in capas:` y `m.add(…Dense(u, activation="relu", …))`: por cada tamaño de la lista añade una capa de u neuronas. `relu` es su regla (si el resultado es negativo lo pone a 0) y `kernel_regularizer=…l2(l2)` activa el castigo L2.
- `m.add(…Dropout(dropout))`: después de cada capa, el *dropout*.
- `m.add(…Dense(1, activation="sigmoid"))`: la capa final: una neurona que da un número entre 0 y 1, la probabilidad de cancelar.
- `m.compile(…)`: dice cómo se entrena: `Adam` es el método para retocar las cuentas, `binary_crossentropy` es la fórmula de la pérdida para un sí/no, y AUC es la nota que se vigila además.
- `modelos_mlp, historiales, tiempos = [], [], []`: tres listas vacías donde se guardarán las redes, su historial y lo que tardan.
- `for semilla in (1, 2, 3):`: repite lo de debajo tres veces, con semillas 1, 2 y 3. Así se ve cuánto depende el resultado de la suerte.
- `m = construir_mlp(A_ent.shape[1])`: construye la red; `A_ent.shape[1]` es el número de columnas (35).
- `m.fit(A_ent, y_ent, validation_data=(A_val, y_val), epochs=40, batch_size=64, …)`: **entrena**. Aprende con entrenamiento, se examina con validación al final de cada época, hace como máximo 40 épocas y retoca las cuentas cada 64 reservas. h guarda el historial.
- `EarlyStopping(monitor="val_loss", patience=6, restore_best_weights=True)`: la parada temprana: si la pérdida de validación no mejora en 6 épocas seguidas, para y deja la red de la **mejor** época.
- `tiempos.append(…)`, `modelos_mlp.append(m)`…: apuntan en las listas lo que ha salido.
- `print(f"semilla …")`: escribe la semilla, cuántas épocas hizo en total (`len(h.history['loss'])`, incluye las 6 últimas de «paciencia») y el mejor AUC de validación que vio.

3. **Mirar las curvas.** Ejecutad la celda de la sección 5. Dibuja tres gráficas (una por semilla) con dos líneas cada una: la pérdida de entrenamiento y la de validación, época a época. En el paso 4 aprenderéis a leerlas; de momento, comprobad que están y que cada una tiene sus dos líneas y su leyenda.
4. **Ejercicio 2: el AUC de cada semilla.** Bajad al enunciado «Ejercicio 2», borrad los `...` de su celda y escribid:

   ```
   aucs_mlp_val = [roc_auc_score(y_val, m.predict(A_val, verbose=0).ravel()) for m in modelos_mlp]
   auc_mlp_medio = float(np.mean(aucs_mlp_val))
   ```

   Qué hace cada línea:

   - `aucs_mlp_val = […for m in modelos_mlp]`: se lee «para cada red `m` de la lista `modelos_mlp`, calcula esto, y guarda todos los resultados en una lista». Salen tres números.
   - `m.predict(A_val, verbose=0)`: pide a la red la probabilidad de cancelar de cada reserva de **validación**. `.ravel()` deja el resultado como una lista plana de números.
   - `roc_auc_score(y_val, …)`: pone nota (AUC) comparando esas probabilidades con lo que pasó de verdad (`y_val`). **Es validación: no uséis `A_pru` ni `y_pru`.**
   - `auc_mlp_medio = float(np.mean(aucs_mlp_val))`: la media de los tres AUC. **Los dos nombres** (`aucs_mlp_val` y `auc_mlp_medio`) **tienen que ser exactamente esos**: el punto de control los usa.

   Pulsad ▶ en la celda.

Qué deberíais ver.

- **El escalado:** la primera línea del `print` es de ceros (pueden salir como `-0.`, que es lo mismo): en entrenamiento la media es 0 porque el escalador aprendió de esas filas. La segunda son números pequeños pero no nulos, aproximadamente `[ 0.02 0.03 0. 0.02 0.01 -0.07 -0.07 -0.01 -0.03]`: validación no es 0 porque el escalador **no la ha visto**.
- **El entrenamiento:** tres líneas como `semilla 1: N épocas, AUC validación = 0.7… (máx.)`, con N como mucho 40 y AUC en torno a 0,7.
- **Las curvas:** tres gráficas con dos líneas cada una. Lo habitual es que la de entrenamiento siga bajando y la de validación baje al principio y luego se aplane o suba un poco.
- **El ejercicio 2:** `AUC de validación por semilla: [0.7… 0.7… 0.7…] · media 0.7…`. El punto de control espera una media entre 0,6 y 0,85. Estos AUC pueden no coincidir del todo con los «(máx.)» de arriba: aquellos eran el mejor de cualquier época; estos son los de la red que se queda.

Hecho cuando: hay un MLP entrenado con su curva de aprendizaje, el ejercicio 2 pasa su comprobación y sabéis decir en una frase qué hace una época.

Si algo falla

Veis…	Qué significa	Qué hacer
`NameError: name 'A_ent' is not defined`	No se ha ejecutado la celda de la sección 1 (o Colab se reinició).	Ejecutad desde la primera celda hacia abajo hasta la sección 1 y volved aquí.
`NameError: name 'modelos_mlp' is not defined`	No se ha ejecutado la celda de la sección 4.	Ejecutadla (tarda unos minutos) y después repetid el ejercicio 2.
`NameError: name 'keras' is not defined`	No se ejecutó la celda de importaciones.	Ejecutad la segunda celda de código y después la de la sección 4.
`AssertionError: Debe haber un AUC por cada uno de los 3 modelos`	La lista no tiene tres números (por ejemplo, solo se ha calculado el de una red).	Usad la forma `[… for m in modelos_mlp]`, que recorre las tres.
`AssertionError: El AUC medio debería rondar 0.7–0.75`	El AUC se ha calculado con datos equivocados (entrenamiento o prueba) o la red no ha aprendido.	Usad `A_val` y `y_val`, y comprobad que no habéis cambiado nada de la celda de la sección 4.
La celda de la sección 4 tarda mucho y no escribe nada	Es normal: entrena tres redes y no muestra progreso (`verbose=0`).	Esperad a que aparezcan las tres líneas `semilla …`. Si tarda mucho más de unos minutos, *Entorno de ejecución → Interrumpir ejecución* y volved a ejecutarla.
Los AUC del ejercicio 2 no coinciden con los «(máx.)» de la sección 4	Son medidas distintas: el «máx.» es el mejor de todas las épocas; el ejercicio mide la red que se queda.	Nada: es normal que difieran en la tercera cifra decimal.

Paso 3 · Variantes (día 2, 25 minutos)

1. Probar **2 o 3 variantes** del MLP cambiando un solo ajuste cada vez.
2. Compararlas mirando **solo la validación**: el test no se toca.
3. Anotar en una tabla pequeña qué ha mejorado y qué no.

En sencillo. Imaginad que estáis ajustando una receta de cocina: más sal, menos tiempo de horno, otra temperatura. Si cambiáis tres cosas a la vez y el plato sale mejor, no sabéis cuál ayudó. Con la red pasa igual: tiene unos «botones» (cuántas neuronas, cuánto *dropout*, cuánta regularización L2, qué tan grandes son los retoques) y vais a probar **un botón cada vez**, siempre con las mismas tres semillas, y a comparar las notas **de validación**. La prueba (el examen final) sigue guardada en el cajón. Puede que ninguna variante mejore, y es un resultado válido: lo importante es apuntarlo con su número. No hace falta saber programar: copiáis una celda y cambiáis **un valor** cada vez.

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
Variante	Una versión del MLP con un ajuste distinto del de la base.
Ajuste (hiperparámetro)	Un «botón» que se fija antes de entrenar: no lo aprende la red, lo elegís vosotros.
Argumento con nombre	Un valor que se pasa a una función escribiendo su nombre: `construir_mlp(35, dropout=0.5)`. Si el nombre está mal escrito, Python da error.
`capas`	Lista con el número de neuronas de cada capa oculta: `(64, 32)` son dos capas. Con una sola capa se escribe `(32,)`: **la coma del final es obligatoria**.
`dropout`	La fracción de neuronas que se apagan al azar en cada paso del entrenamiento (0.3 = el 30 %).
`l2`	La fuerza del castigo a los pesos grandes. Más L2 = red más contenida.
`tasa`	La tasa de aprendizaje: el tamaño de cada retoque. Muy grande, la red se pasa de largo; muy pequeña, tarda mucho.
Ruido entre semillas	Lo que cambia el resultado solo por la suerte de la semilla. Una mejora más pequeña que ese vaivén no cuenta.

Cómo se hace

1. **Crear una celda para las variantes.** Justo después del ejercicio 2 (que ya debe pasar su comprobación), pulsad **+ Código** y pegad esta celda. Entrena la red con las tres semillas de siempre, pero guarda los resultados en listas **nuevas** (acabadas en _v) para no pisar `modelos_mlp` ni `historiales`:

```
etiqueta = "base"
perd_ent_v, perd_val_v, aucs_v = [], [], []
for semilla in (1, 2, 3):
    keras.utils.set_random_seed(semilla)
    m = construir_mlp(A_ent.shape[1], capas=(64, 32), l2=0.003, dropout=0.3, tasa=1e-3)
    h = m.fit(A_ent, y_ent, validation_data=(A_val, y_val), epochs=40, batch_size=64, verbose=0,
              callbacks=[keras.callbacks.EarlyStopping(monitor="val_loss", patience=6, restore_best_weights=True)])
    mejor = int(np.argmin(h.history["val_loss"]))
    perd_ent_v.append(h.history["loss"][mejor])
    perd_val_v.append(h.history["val_loss"][mejor])
    aucs_v.append(roc_auc_score(y_val, m.predict(A_val, verbose=0).ravel()))
    print(f"semilla {semilla}: mejor época {mejor + 1} de {len(h.history['loss'])}")
print(f"{etiqueta}: pérdida entrenamiento {np.mean(perd_ent_v):.3f} · pérdida validación {np.mean(perd_val_v):.3f} · AUC
validación {np.mean(aucs_v):.3f} ± {np.std(aucs_v):.3f}")
```

   Qué hace cada línea:

   - `etiqueta = "base"`: un nombre para reconocer la prueba en lo que se escribe al final. Lo cambiaréis en cada variante.
   - `perd_ent_v, perd_val_v, aucs_v = [], [], []`: tres listas vacías donde se apuntará, por semilla, la pérdida de entrenamiento, la de validación y el AUC de validación.
   - `for semilla in (1, 2, 3):` y `keras.utils.set_random_seed(semilla)`: igual que en el paso 2: tres vueltas, cada una con su semilla.
   - `m = construir_mlp(A_ent.shape[1], capas=(64, 32), l2=0.003, dropout=0.3, tasa=1e-3)`: construye la red **escribiendo los cuatro ajustes**. Estos valores son los de la base. **Aquí es donde cambiáis un valor por variante** (y solo uno).
   - `h = m.fit(…)`: entrena exactamente igual que en el paso 2 (mismos datos, 40 épocas como máximo, parada temprana).
   - `mejor = int(np.argmin(h.history["val_loss"]))`: busca en qué época la pérdida de validación fue la más baja (`np.argmin` da la posición del valor más pequeño, contando desde 0). Esa es la época de la red que se queda, porque `restore_best_weights=True`.
   - `perd_ent_v.append(h.history["loss"][mejor])` y `perd_val_v.append(h.history["val_loss"][mejor])`: apuntan la pérdida de entrenamiento y la de validación **en esa mejor época**.
   - `aucs_v.append(roc_auc_score(y_val, m.predict(A_val, verbose=0).ravel()))`: el AUC de validación de esa red, como en el ejercicio 2.
   - `print(f"semilla …")`: escribe en qué época estaba lo mejor (`mejor + 1`, contando desde 1) y cuántas épocas hizo en total.
   - El `print` final: la media de las tres semillas de cada cifra y, tras el ±, la desviación del AUC entre semillas (`np.std`), que es el ruido entre semillas.

2. **Ejecutar primero la base.** Con `etiqueta = "base"` y los valores tal cual, pulsad ▶. Tarda unos minutos como mucho. Apuntad la línea final: es la primera fila de la tabla y es **vuestra regla de medir** para las variantes (se mide igual que ellas).

3. **Probar 2 o 3 variantes, cambiando un solo valor.** Copiad la celda, cambiad `etiqueta` y **un único** valor de la línea de `construir_mlp`, y ejecutad. Ideas:

   - Menos neuronas: `capas=(32,)` (con la coma).
   - Más neuronas: `capas=(128, 64)`.
   - Más *dropout*: `dropout=0.5`.
   - Más L2: `l2=0.01`.
   - Retoques más pequeños: `tasa=3e-4` (se lee 0,0003).

   Si usáis otra paciencia en la parada temprana, cambiad `patience=6` por otro valor. Ejemplo de una variante: `etiqueta = "dropout 0.5"` y `m = construir_mlp(A_ent.shape[1], capas=(64, 32), l2=0.003, dropout=0.5, tasa=1e-3)`.

4. **Rellenar la tabla.** Una fila por prueba, con las cifras de la última línea de cada ejecución:

Variante	Qué cambia	Error en entrenamiento	Error en validación	AUC en validación
Base	`capas=(64, 32)`, `dropout=0.3`, `l2=0.003`	(vuestra cifra)	(vuestra cifra)	(vuestra cifra)
Variante 1	(lo que cambiáis)	(vuestra cifra)	(vuestra cifra)	(vuestra cifra)
Variante 2	(lo que cambiáis)	(vuestra cifra)	(vuestra cifra)	(vuestra cifra)

   Añadid, por variante, una frase con el número: «mejoró» (cuánto), «empeoró» o «igual». Cuidado con L2: la pérdida incluye el castigo, así que con más L2 la pérdida **sube** aunque la red no sea peor. Entre variantes, comparad sobre todo el **AUC**.

5. **Elegir.** Quedaos con la variante de **mayor AUC en validación**. Si dos quedan casi iguales (la diferencia es menor que el ± de cualquiera de ellas), es ruido: escoged la más sencilla (menos neuronas, menos capas). **Si miráis la prueba para decidir, ya no es una prueba.** Escribid en el documento del equipo cuál elegís y por qué.

6. **Dejar la elegida en el cuaderno.** Si la elegida no es la base, cambiad en la celda de la **sección 4** (la del paso 2) la línea `m = construir_mlp(A_ent.shape[1])` añadiendo el ajuste, por ejemplo `m = construir_mlp(A_ent.shape[1], dropout=0.5)`. Después ejecutad otra vez, **en orden y una a una**, la celda de la sección 4, la del ejercicio 2 y la de las curvas (sección 5). **No ejecutéis todavía la sección 6**: usa la prueba. Así los pasos 4 y 5 trabajan con la variante elegida.

Qué deberíais ver. Por cada ejecución, tres líneas `semilla N: mejor época X de Y` y una línea final con las cuatro cifras (por ejemplo, `base: pérdida entrenamiento 0.… · pérdida validación 0.… · AUC validación 0.7… ± 0.0…`). Al final, una tabla con la base y las 2–3 variantes y una frase por variante. Es muy posible que las diferencias de AUC sean pequeñas. Que una variante no mejore es un resultado válido: lo importante es apuntarlo.

Hecho cuando: tenéis la tabla rellena, habéis elegido una variante mirando solo la validación, sabéis explicar por qué y el cuaderno (sección 4) usa la elegida.

Si algo falla

Veis…	Qué significa	Qué hacer
`NameError: name 'construir_mlp' is not defined` (o `'A_ent'`, `'keras'`)	No se han ejecutado las celdas de las secciones 1 o 4.	Ejecutad desde arriba hasta la sección 4 y después la celda de variantes.
`TypeError: construir_mlp() got an unexpected keyword argument 'dropuot'`	Un nombre de ajuste está mal escrito.	Los nombres son `capas`, `l2`, `dropout` y `tasa`.
`TypeError: 'int' object is not iterable`	Habéis escrito `capas=32` en lugar de `capas=(32,)`.	Poned paréntesis y la coma del final.
La variante da exactamente los mismos números que la base	No ha cambiado ningún valor (o solo cambió `etiqueta`).	Cambiad el valor dentro de `construir_mlp(…)`, no solo el nombre.
Con `l2=0.01` la pérdida es más alta que en la base	La pérdida incluye el castigo L2. No quiere decir que la red sea peor.	Comparad por el AUC de validación.
El AUC sube 0.002 y no sabéis si cuenta	Es menos que el ruido entre semillas (el ±).	Contadlo como «igual» y quedaos con la variante más sencilla.
La celda tarda mucho	Entrena tres redes cada vez.	Probad menos variantes (basta con 2) y esperad a las tres líneas `semilla ….`

Paso 4 · Diagnosticar el sobreajuste (día 2, 20 minutos)

1. Mirar las curvas de vuestra mejor red.
2. Hacer el ejercicio 3 (`diagnostico`).
3. Escribir una frase de diagnóstico.

En sencillo. Un alumno que memoriza las respuestas del libro de ejercicios saca un 10 en el libro y un 5 en el examen: se ha aprendido el libro, no la materia. Una red hace lo mismo cuando **memoriza** las reservas de entrenamiento y falla con las nuevas: se llama **sobreajuste**. Las curvas de aprendizaje son esas dos notas, la del libro (entrenamiento) y la del examen de prueba (validación), época a época, y dejan ver si la red memoriza, si no ha aprendido lo bastante o si va bien. En este paso aprendéis a leer esas curvas, escribís una pequeña función que lo decide y contáis el resultado con una frase. **No hace falta** saber matemáticas: solo comparar dos líneas.

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
Sobreajuste	La red memoriza el entrenamiento y falla con datos nuevos: la pérdida de entrenamiento baja, pero la de validación se estanca o sube.
Subajuste	La red es demasiado simple (o ha entrenado poco) y falla en todo: las dos pérdidas se quedan altas.
Buen ajuste	Las dos pérdidas bajan y se estabilizan juntas, una cerca de la otra.
Historial (`History`)	El objeto que devuelve `fit`. `h.history` es un diccionario con las pérdidas de cada época: la clave `"loss"` es la de entrenamiento y `"val_loss"` la de validación.
Brecha	La distancia entre la pérdida de validación y la de entrenamiento. Una brecha grande avisa de memorización.
Margen	Cuánto se tolera que la validación final se pase de su mejor valor antes de llamarlo sobreajuste (aquí 0,03).
Condición (`if`)	Una línea que ejecuta lo que tiene debajo **solo si** se cumple lo que dice. `or` significa «o»: basta con que se cumpla una de las dos cosas.
`[-1]`	El último elemento de una lista. `pv[-1]` es la pérdida de la última época.

Cómo se hace

1. **Mirar las curvas.** Si no las tenéis a la vista, ejecutad otra vez la celda de la sección 5 (con la red elegida en el paso 3). Hay una gráfica por semilla, con la pérdida de entrenamiento y la de validación. Cuidado: el eje horizontal empieza en 0, así que la primera época es el 0 del dibujo. Buscad cuál de estas tres formas se parece más:
   - **El entrenamiento baja y la validación sube** (o se aleja hacia arriba): la red memoriza (sobreajuste). Se arregla con más regularización, menos neuronas o parando antes.
   - **Las dos están altas y juntas:** la red es demasiado simple (subajuste).
   - **Las dos bajan y se estabilizan juntas:** buen ajuste.

   Para saber en qué época la validación fue mejor, ejecutad en una celda nueva (**+ Código**):

   ```
   for s, h in zip((1, 2, 3), historiales):
       print(f"semilla {s}: la pérdida de validación es mínima en la época {int(np.argmin(h.history['val_loss'])) + 1} de
   {len(h.history['val_loss'])}")
   ```

   `zip((1, 2, 3), historiales)` empareja cada semilla con su historial; `np.argmin(…)` da la posición de la pérdida de validación más baja (contando desde 0) y el `+ 1` la pasa a «contando desde 1». `len(…)` es el número de épocas que se hicieron.

2. **Ejercicio 3: la función `diagnostico`.** Bajad al enunciado «Ejercicio 3» (sección 5). En su celda hay un `# TODO`, tres puntos `...` y varias líneas con `assert`. Borrad **solo los tres puntos** y escribid:

   ```
   def diagnostico(historial, margen=0.03):
       pv, pe = historial.history["val_loss"], historial.history["loss"]
       if pv[-1] > min(pv) + margen or (pv[-1] - pe[-1]) > 0.15:
           return "sobreajuste"
       return "sin sobreajuste claro"
   ```

   Qué hace cada línea:

   - `def diagnostico(historial, margen=0.03):`: define la función. Recibe un historial de Keras y un `margen` (si no se dice, 0,03). No olvidéis los dos puntos del final.
   - `pv, pe = historial.history["val_loss"], historial.history["loss"]`: guarda en `pv` la lista de pérdidas de validación y en `pe` la de entrenamiento, una cifra por época. Los nombres `"val_loss"` y `"loss"` van entre comillas y **no se pueden cambiar**.
   - `if pv[-1] > min(pv) + margen or (pv[-1] - pe[-1]) > 0.15:`: la condición. Primera parte: ¿la pérdida de validación **final** (`pv[-1]`) supera a su **mínimo** (`min(pv)`) en más de `margen`? Segunda parte: ¿la validación final supera a la de entrenamiento en más de 0,15 (la brecha)? Con que se cumpla una de las dos, hay sobreajuste.
   - `return "sobreajuste"`: va **con 8 espacios** (está dentro del `if`) y devuelve esa palabra, escrita tal cual.
   - `return "sin sobreajuste claro"`: va con 4 espacios y es lo que se devuelve si la condición no se cumple.

   Pulsad ▶. Los `assert` prueban la función con dos historiales de juguete, y la celda aplica `diagnostico` a las tres semillas por sí sola.

3. **Escribir la frase de diagnóstico.** En una celda de texto (**+ Texto**) junto a la gráfica, y resumida luego en el informe del paso 6, una frase para vuestra mejor red con una cifra o una época. Molde: *«Con la variante [X], la validación deja de mejorar en la época [N] y el entrenamiento sigue bajando: [hay un poco de sobreajuste, que la parada temprana corta a tiempo / no hay sobreajuste claro: las dos curvas bajan juntas]»*. Tiene que decir qué **veis** en vuestras curvas, no lo que «suele» pasar.

Qué deberíais ver. Tres líneas `semilla N: la pérdida de validación es mínima en la época X de Y`; tres líneas `MLP semilla 1: …` (y 2 y 3), cada una con `sobreajuste` o `sin sobreajuste claro`; y vuestra frase escrita junto a la gráfica. No hay una respuesta «correcta»: depende de vuestras curvas. Ojo: `diagnostico` mira la **última** época, que suele ser seis épocas posterior a la mejor (la parada temprana espera ese tiempo). Por eso puede salir `sobreajuste` aunque el cuaderno se quede con la mejor versión: en ese caso, contadlo así.

Hecho cuando: el ejercicio 3 pasa su comprobación y la frase de diagnóstico está escrita (en el cuaderno y, resumida, en el informe).

Si algo falla

Veis…	Qué significa	Qué hacer
`NameError: name 'diagnostico' is not defined`	La celda del ejercicio 3 aún tiene los `...` o no se ha ejecutado.	Escribid la función y volved a pulsar ▶.
`IndentationError: expected an indented block…`	Las líneas de dentro de la función (o del `if`) no tienen sangría.	4 espacios dentro de `def` y 8 en el `return` que cuelga del `if`.
`AssertionError: Validación creciente y brecha grande = sobreajuste`	La función no devuelve `"sobreajuste"` cuando la validación sube mucho.	Revisad la condición: `pv[-1] > min(pv) + margen`, el `or` y `> 0.15`.
`AssertionError: Curvas paralelas y juntas: sin sobreajuste`	La función dice `"sobreajuste"` con curvas buenas.	Comprobad que la brecha es `pv[-1] - pe[-1]` (validación menos entrenamiento) y que el 0.15 no se ha cambiado.
`AttributeError: 'list' object has no attribute 'history'`	Se ha pasado la lista `historiales` entera en vez de un historial.	Llamad a `diagnostico(h)` con uno de los tres (o dentro del `for` de la celda).
`AttributeError: 'dict' object has no attribute 'history'`	Se ha pasado `h.history` (el diccionario) en lugar de `h`.	Pasad `h`, sin el `.history`: la función ya lo añade.
Las tres semillas dicen `sobreajuste`	`diagnostico` mira la última época, que viene tras la «paciencia» de 6 épocas.	No es un fallo: miradlo en la gráfica y contadlo como «empezó a memorizar tras la época N; la parada temprana nos devuelve el mejor punto».

Paso 5 · Comparación justa (día 2, 25 minutos)

1. Repetir el entrenamiento con varias semillas y calcular media ± desviación.
2. Medir el veredicto en la prueba **una sola vez**, con el MLP elegido y con la referencia.
3. Hacer el ejercicio 4 (¿la diferencia es real o es ruido?).
4. Añadir el coste: tiempo, complejidad y explicabilidad.

En sencillo. Es el momento del veredicto, el examen final. Se juega **una sola vez**: la prueba lleva guardada desde el paso 1 y, si la mirarais para retocar la red, dejaría de ser un examen justo. Se enfrentan el boosting y el MLP (la media de las tres redes) y se comparan con las mismas notas. Pero un 0,74 frente a un 0,73 puede ser pura suerte, así que el cuaderno hace un truco llamado *bootstrap*: repite 300 veces el examen con reservas sorteadas (con repetición) y mira cuánto baila la diferencia entre los dos modelos. Si el vaivén incluye el cero, **no podéis decir que uno gane**: es un empate, y es una respuesta válida. Por último se pone el precio: cuánto tardan, qué tan complicados son y si se pueden explicar. No hace falta entender cómo se calcula el bootstrap: basta con ejecutarlo y leer el intervalo.

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
Prueba (*test*)	El 20 % de las reservas que ningún modelo ha visto ni se ha usado para decidir. Se mira una vez, al final.
Veredicto	La nota final de cada modelo en la prueba.
Media ± desviación	La media de varias medidas y, tras el ±, cuánto se separan de ella. Mide el ruido entre semillas.
Bootstrap	Repetir el cálculo muchas veces con reservas sorteadas con repetición para ver cuánto cambia el resultado por azar.

Palabra	Qué significa, en sencillo
Intervalo del 95 %	El tramo de valores `[inf, sup]` en el que cae el 95 % de los resultados del bootstrap. Si incluye el 0, la diferencia no está clara.
Diferencia de AUC	El AUC del MLP menos el AUC del boosting. Positiva: gana la red; negativa: gana el boosting; cerca de 0: parecidos.
Empate	Cuando el intervalo incluye el 0: los datos no bastan para decir cuál es mejor.
Booleano	Un valor que solo puede ser `True` o `False`.
Explicabilidad	Lo fácil que es contar **por qué** un modelo dice «cancelará» a una reserva concreta.

Cómo se hace

1. **Calcular media ± desviación.** Comprobad primero que el cuaderno usa el MLP elegido en el paso 3 (que se han ejecutado la sección 4 y el ejercicio 2 con él). En una celda nueva:

   ```
   print(f"AUC de validación del MLP: {np.mean(aucs_mlp_val):.3f} ± {np.std(aucs_mlp_val):.3f}")
   ```

   `np.mean(aucs_mlp_val)` es la media de los tres AUC de validación del ejercicio 2; `np.std(…)` es la desviación (cuánto se separan los tres de esa media) y `:.3f` los deja con tres decimales. El boosting entrena siempre igual (tiene `random_state` fijo), así que no tiene desviación entre semillas: su incertidumbre la da el bootstrap.

2. **El veredicto en la prueba.** Ejecutad la celda «6 · El veredicto en el conjunto de prueba». Ya está escrita y usa la función `evaluar` del ejercicio 1. **Es la única vez que se usa la prueba para decidir: no cambiéis el MLP después de verla.**

   ```
   prob_mlp_pru = np.mean([m.predict(A_pru, verbose=0).ravel() for m in modelos_mlp], axis=0)
   tabla = pd.DataFrame({"Boosting de árboles": evaluar(y_pru, prob_boost_pru),
                         "MLP (media de 3 semillas)": evaluar(y_pru, prob_mlp_pru)}).T
   tabla["entrenamiento (s)"] = [t_boost, np.mean(tiempos)]
   tabla["parámetros / árboles"] = ["200 iteraciones × árboles de profundidad ≤ 4", f"{modelos_mlp[0].count_params():,} pesos"]
   tabla.round(3)
   ```

   Qué hace cada línea:

   - `prob_mlp_pru = np.mean([… for m in modelos_mlp], axis=0)`: cada una de las tres redes da su probabilidad para cada reserva de **prueba** y se hace la media de las tres: un pequeño «comité».
   - `tabla = pd.DataFrame({…}).T`: monta una tabla con dos columnas, una por modelo, y cada una con las cuatro notas de `evaluar`; `.T` la gira para que cada modelo sea una fila.
   - `tabla["entrenamiento (s)"] = […]`: añade lo que tardó el boosting (`t_boost`) y lo que tardó el MLP de media (`np.mean(tiempos)`).
   - `tabla["parámetros / árboles"] = […]`: añade el tamaño: las iteraciones del boosting y el número de pesos de la red (`count_params()`).
   - `tabla.round(3)`: muestra la tabla con tres decimales.

3. **¿Real o ruido? El bootstrap.** Ejecutad la celda siguiente, también escrita:

```
rng = np.random.default_rng(SEMILLA)
difs = []
for _ in range(300):
    idx = rng.integers(0, len(y_pru), len(y_pru))
    if len(np.unique(y_pru[idx])) < 2:
        continue
    difs.append(roc_auc_score(y_pru[idx], prob_mlp_pru[idx]) - roc_auc_score(y_pru[idx], prob_boost_pru[idx]))
inf, sup = np.percentile(difs, [2.5, 97.5])
print(f"Diferencia de AUC (MLP − boosting): {np.mean(difs):+.3f} · intervalo 95 %: [{inf:+.3f}, {sup:+.3f}]")
plt.hist(difs, bins=25, color="steelblue"); plt.axvline(0, color="red", ls="--", label="sin diferencia")
```

   Qué hace cada parte (el final de la celda solo pone títulos a los ejes y muestra el dibujo):

   - `for _ in range(300)`:: repite 300 veces lo de debajo.
   - `idx = rng.integers(0, len(y_pru), len(y_pru))`: sortea 1200 posiciones entre las 1200 reservas de prueba, **con repetición**: una prueba «remuestreada».
   - `if len(np.unique(y_pru[idx])) < 2: continue`: si en el sorteo solo hubiera un tipo de reserva no se puede calcular el AUC, y se pasa a la vuelta siguiente.
   - `difs.append(…)`: calcula el AUC del MLP menos el del boosting **en esa prueba sorteada** y lo apunta en `difs`.
   - `inf, sup = np.percentile(difs, [2.5, 97.5])`: se queda con el 95 % central de las 300 diferencias: `inf` es el extremo de abajo y `sup`, el de arriba. Ese es el **intervalo del 95 %**.
   - `print(f"…")`: escribe la diferencia media y el intervalo (`:+.3f` pone el signo).
   - `plt.hist(…)` y `plt.axvline(0, …)`: dibujan el histograma de las 300 diferencias y una línea roja discontinua en el 0.

4. **Ejercicio 4: ¿hay diferencia?** Bajad a «Ejercicio 4», borrad los `...` de su celda y escribid:

```
hay_diferencia = bool(inf > 0 or sup < 0)
mejor_modelo = ("MLP" if np.mean(difs) > 0 else "Boosting") if hay_diferencia else "Empate"
```

   Qué hace cada línea:

   - `hay_diferencia = bool(inf > 0 or sup < 0)`: el intervalo **no contiene el 0** cuando está entero por encima (`inf > 0`) o entero por debajo (`sup < 0`). `bool(…)` lo convierte en un `True`/`False` de Python: **sin `bool(…)` el `assert` falla**, porque NumPy devuelve su propio tipo de verdadero/falso.
   - `mejor_modelo = (…) if hay_diferencia else "Empate"`: se lee «si hay diferencia, … ; si no, `"Empate"`». Con diferencia, gana `"MLP"` si la diferencia media es positiva y `"Boosting"` si no. Las tres palabras se escriben tal cual, con mayúscula inicial.

   Pulsad ▶: la celda imprime `Veredicto estadístico: …`.

5. **Apuntar el coste.** Con la tabla delante, rellenad una mini tabla para el informe con tres filas: **tiempo** (la columna `entrenamiento (s)`), **complejidad** (árboles o número de pesos) y **explicabilidad**. Para esta última, escribid con vuestras palabras si os parece más fácil o más difícil explicar a un directivo por qué se cancela una reserva con cada modelo (en general, un boosting es más fácil de explicar que una red). Un ejemplo de frase: «*La red tarda [N] veces más y mejora el AUC en [X]; no compensa si el intervalo incluye el 0*».

Qué deberíais ver.

- **Media ± desviación:** una línea como `AUC de validación del MLP: 0.7… ± 0.0…`.
- **La tabla de la sección 6:** dos filas, `Boosting de árboles` y `MLP (media de 3 semillas)`, con las columnas AUC, AP, F1, exactitud, entrenamiento (s) y parámetros / árboles. Para el boosting, con esta partición, salen aproximadamente AUC 0.738, AP 0.722, F1 0.639 y exactitud 0.672 (un modelo que dijera siempre «no cancela» acertaría un 54 %). El MLP debería quedar muy cerca, con un tiempo de entrenamiento bastante mayor y, con las capas por defecto, `4.417 pesos` (35·64 + 64 + 64·32 + 32 + 32 + 1).
- **El bootstrap:** una línea `Diferencia de AUC (MLP − boosting): +0.0… · intervalo 95 %: [-0.0…, +0.0…]` y un histograma con una línea roja discontinua en el 0.
- **El ejercicio 4:** `Veredicto estadístico: Empate` (o `MLP`, o `Boosting`). Con estos datos lo esperable es que los dos queden muy cerca, y un «Empate» es un resultado perfectamente válido.

Hecho cuando: el ejercicio 4 pasa su comprobación, tenéis media ± desviación (o el intervalo) de ambos enfoques y los números del coste apuntados.

Si algo falla

Veis…	Qué significa	Qué hacer
`AssertionError: hay_diferencia debe ser True/False`	El valor es de tipo NumPy, no un `True`/`False` de Python.	Envolvedlo: `hay_diferencia = bool(inf > 0 or sup < 0)`.
`AssertionError: mejor_modelo debe ser 'MLP', 'Boosting' o 'Empate'`	Hay una errata o una minúscula (`"mlp"`, `"empate"`).	Escribid exactamente `"MLP"`, `"Boosting"` o `"Empate"`.
`AssertionError: Si no hay diferencia clara debe ser 'Empate' (y viceversa)`	`mejor_modelo` y `hay_diferencia` no dicen lo mismo.	Con `hay_diferencia` en `False`, `mejor_modelo` ha de ser `"Empate"`; y con `True`, `"MLP"` o `"Boosting"`.
`NameError: name 'inf' is not defined` (o `'sup'`, `'difs'`)	No se ejecutó la celda del bootstrap.	Ejecutadla y después repetid el ejercicio 4.
`NameError: name 'prob_mlp_pru' is not defined` (o `'prob_boost_pru'`)	No se ejecutó la celda de la tabla (o la del boosting del paso 1).	Ejecutad en orden: boosting, tabla de la sección 6, bootstrap, ejercicio 4.
Las cifras cambian un poco al volver a ejecutar todo	El entrenamiento de una red no es idéntico al 100 % entre ejecuciones y ordenadores.	Si la diferencia es pequeña, es normal: usad las cifras de la ejecución final (la de *Reiniciar y ejecutar todo*) en el informe.
Queréis cambiar el MLP después de ver la tabla	Eso usaría la prueba para decidir.	No lo hagáis. Anotad lo que salió; lo que probaríais es un «siguiente paso» para el informe.

Paso 6 · Informe, revisión y entrega (día 2, 35 minutos)

1. Escribir el informe de 1 página, con la recomendación al principio.
2. Rellenar el README de `sprint-10/`.
3. Pedir a alguien que no escribió esa parte que revise con la rúbrica y anote el resultado al final del informe.
4. Subir los tres ficheros del equipo a `sprint-10/`, y cada persona su autoevaluación.

En sencillo. Es el paso de **cerrar el trabajo**. Primero, contar lo que habéis descubierto como se lo contaríais a la dirección de TurisData: en una página y **empezando por la respuesta**, igual que un titular de periódico («Recomendamos… porque…») y después los detalles que la sostienen. Segundo, comprobar que otra persona puede repetir vuestro cuaderno y que alguien con ojos frescos lo revisa con la rúbrica. Tercero, subir todo al repositorio del equipo desde el navegador. No hay código nuevo. Recordad que un «no compensa» bien demostrado es tan válido como un «sí».

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
Informe comparativo	El documento de 1–2 páginas que compara el modelo clásico con la red y termina en una recomendación.
Markdown (`.md`)	Texto normal con unas pocas marcas: `#` para un título, `**…**` para negrita y barras verticales para las tablas. GitHub lo muestra ya con formato.
Riesgo de IA responsable	Algo concreto que podría salir mal si se usara el modelo (por ejemplo, datos inventados o un grupo de clientes peor tratado), junto con una **medida** para evitarlo.
README	La hoja de entrada de la carpeta `sprint-10/`: qué hay, quién lo hizo y cómo se repite. Es lo primero que lee el docente.
Commit	Cada vez que guardáis un cambio en GitHub con el botón verde **Commit changes**. Queda apuntado con el nombre de quien lo hace.
Rúbrica	La tabla con la que se evalúa el reto: cuatro criterios, cada uno de 1 a 4.
Reiniciar y ejecutar todo	Borra la memoria del cuaderno y ejecuta todas las celdas de arriba abajo. Prueba que el cuaderno funciona solo y deja los resultados visibles.

Cómo se hace

1. **Repartir el trabajo.** Cada persona sube lo suyo con su cuenta: así el historial muestra a todo el equipo, que es lo que se evalúa. Un reparto posible: una persona prepara y sube el cuaderno, otra escribe el informe y otra rellena el README. Quien revise con la rúbrica (el punto 5 de esta lista) **no puede ser quien escribió esa parte**.

2. **Escribir el informe** `informe_comparativo.md`. Abrid `plantilla_informe_comparativo.md`, pulsad *Raw*, copiad el texto y, en la carpeta `sprint-10` del repositorio, usad *Add file* → *Create new file* con el nombre `informe_comparativo.md` y pegadlo. Tiene seis partes **en este orden**; sustituid lo que va entre corchetes por vuestras cifras y frases.

   De dónde sale cada parte:

Parte	Qué escribís	De dónde sale
1. Pregunta y respuesta	Una frase: «Recomendamos mantener / cambiar a… porque…»	Vuestra decisión del paso 5
2. Resultados	AUC, AP y F1 de la tabla de la sección 6; media ± desviación del MLP; el intervalo del bootstrap	Pasos 2 y 5
3. Curva comentada	Qué se ve y vuestra frase de diagnóstico	Paso 4
4. Coste	Tiempo, complejidad y explicabilidad	Paso 5
5. Riesgos de IA responsable	Al menos dos, específicos, cada uno con su medida	Vuestra reflexión
6. Límites	Qué no sabemos y qué haríamos después	Vuestra reflexión

   Un ejemplo de cómo suena la parte 1 (con huecos; si vuestros números dicen otra cosa, escribid otra cosa): *«Recomendamos mantener el boosting porque la red no lo mejora de forma clara en la prueba (diferencia de AUC de [X], con un intervalo que incluye el 0), tarda [N] veces más en entrenar y es más difícil de explicar»*.

   Los riesgos tienen que ser **específicos de este caso**, no frases generales como «la IA puede tener sesgos». Dos ejemplos del tipo que se pide: *«Riesgo: los datos son sintéticos (inventados), así que el resultado puede no valer con reservas reales. Medida: repetir la comparación con datos reales del hotel antes de cambiar nada»*; *«Riesgo: hay 138 reservas sin país (el cuaderno las trata como un país más) y el modelo puede rendir peor con los países poco frecuentes. Medida: medir el AUC por país antes de usarlo»*. Pensad también en el umbral y en lo que cuesta equivocarse: cancelar de más o de menos provoca *overbooking* o habitaciones vacías.

   No vale: un informe sin recomendación, cifras escritas de memoria o copiadas de lo que dijo una IA (todas salen de vuestro cuaderno), ni riesgos genéricos.

3. **Guardar la curva (opcional).** Si queréis ver la curva dentro del informe, guardad la imagen de la gráfica del cuaderno con el nombre `curvas_aprendizaje.png` (clic derecho sobre la gráfica → *Guardar imagen como…*) y subidla a `sprint-10/`. Añadid también la línea de **uso de la IA** (parte final de la plantilla): qué pedisteis, qué comprobasteis a mano y qué cambiasteis.

4. **Rellenar el README.** Abrid `sprint-10/README.md` en el repositorio del equipo → icono del lápiz y completad los cuatro campos que ya trae. Así puede quedar:

```
# Sprint 10 · Entregable

- **Reto:** ¿Vale la pena la red?
- **Qué se entrega:** el cuaderno `reto_vale_la_pena.ipynb` y el informe `informe_comparativo.md`
- **Quién hizo qué:**
  - Persona 1 (@usuario): cuaderno y ejercicios 1 y 2
  - Persona 2 (@usuario): variantes, diagnóstico y ejercicio 3
  - Persona 3 (@usuario): comparación, ejercicio 4 e informe
- **Cómo reproducirlo:** abrir el cuaderno en Colab y ejecutar de arriba abajo (*Entorno de ejecución → Reiniciar y
ejecutar todo*)
```

   Después pulsad **Commit changes**.

5. **La revisión con la rúbrica (5 minutos).** Quien revisa **no escribió esa parte**. Sigue los cinco pasos de «Cómo se revisa» de entrega y rúbrica: reiniciar y ejecutar el cuaderno, leer el informe como si fuera el cliente, mirar el cuaderno, poner un nivel de 1 a 4 por criterio **con una frase de prueba** y apuntar el resultado. Rellena la sección «Revisión con la rúbrica» que la plantilla trae al final del informe:

   Un ejemplo de frase de prueba: «3 · reinicia y ejecuta sin errores y el AUC coincide con el informe, pero falta explicar el coste». Sin frase, el nivel no vale. Si hay tiempo, arreglad lo más fácil; lo demás queda como «lo primero que mejoraríamos». Si sobran minutos, empezad un extra (la variante de precio, o guardar el modelo y recargarlo).

6. **Subir los ficheros (la vía del curso, desde el navegador).** Seguid Cómo entregar en GitHub. Para el cuaderno: en Colab, *Entorno de ejecución → Reiniciar y ejecutar todo*, mirad que ninguna celda dé error y después *Archivo → Descargar → Descargar .ipynb*; renombrad el archivo descargado a `reto_vale_la_pena.ipynb` (minúsculas, sin espacios ni tildes). En el repositorio del equipo: (1) **entrad primero en la carpeta `sprint-10`**; (2) *Add file → Upload files*; (3) arrastrad el fichero; (4) escribid un mensaje corto, por ejemplo `Añado el cuaderno con los resultados`, y pulsad **Commit changes**. El informe también se puede crear directamente con *Add file → Create new file*. Cada subida es un *commit* con el nombre de quien la hace, así que repartid las subidas. El CSV va a `data/` solo si aún no está.
7. **La autoevaluación.** Cada persona copia la plantilla de autoevaluación y coevaluación, la guarda con su nombre (`autoevaluacion_nombre.md`, sin espacios ni tildes), la rellena y la sube a `sprint-10/`. La coevaluación va dentro del mismo fichero y se hace durante la *review*.

Qué deberíais ver. En la carpeta `sprint-10` del repositorio, el cuaderno con sus resultados visibles, el informe, el README relleno y una hoja de autoevaluación por persona (**3 + N**); y en el historial (el icono del reloj), *commits* de personas distintas.

Hecho cuando: otra persona podría repetir el cuaderno siguiendo solo el README, el informe termina con «Recomendamos… porque…» y trae la sección «Revisión con la rúbrica» rellena, y los ficheros están en `sprint-10/`.

Después, en la entrega y la review: cada persona rellena y sube su autoevaluación y coevaluación a `sprint-10/`, y el equipo enseña, en unos 5 minutos, la tabla comparativa, una curva de aprendizaje comentada y la recomendación en una frase, con el coste y los riesgos.

Si algo falla

Veis…	Qué significa	Qué hacer
El archivo ha caído en la raíz del repositorio y no en `sprint-10/`	Se subió sin entrar antes en la carpeta.	Abrid el archivo → icono del lápiz → en la ruta de arriba escribid `sprint-10/` delante del nombre → **Commit changes**. Lo mueve.
El archivo no aparece tras subirlo	Falta pulsar el botón verde **Commit changes**: sin él no se guarda nada.	Volved a subirlo y pulsadlo.
No veis el botón *Add file*	No sois colaboradores del repositorio o no habéis aceptado la invitación.	Pedid a quien creó el repositorio que os añada y aceptad la invitación.
El cuaderno subido no muestra gráficas ni cifras	Se descargó sin haberlo ejecutado.	*Entorno de ejecución → Reiniciar y ejecutar todo* y descargad de nuevo el `.ipynb`.
Reiniciar y ejecutar todo se para en una celda con `...`	Falta rellenar el hueco de un ejercicio.	Completad los ejercicios 1 a 4, en orden, y repetid.
Aviso de conflicto al guardar	Dos personas editaron el mismo archivo a la vez.	Recargad la página y repetid vuestro cambio sobre la versión actual.
El informe termina en cifras y no dice qué hacer	Falta la recomendación.	Añadid, al principio, «Recomendamos… porque…».

Ayuda 1 · Cómo usar la IA en este reto

La IA puede ahorrar tiempo, pero **explica con mucha seguridad cosas que pueden estar mal**. Reglas:

- **Solo se compara con las mismas métricas y la misma partición.** No vale cambiar la referencia para que la red gane.
- **Las variantes se eligen con la validación**, nunca con el test.
- **Todo lo que dé, lo ejecutáis y lo entendéis.** Si el docente pregunta por una celda, tenéis que saber explicarla.
- **Declaradlo** en el informe: dónde habéis usado IA y qué habéis comprobado. **Nada de datos personales ni claves** en una IA.

Para qué	Ejemplo de petición	Qué comprobáis
Entender conceptos	«Explícame qué es el sobreajuste en una red neuronal con una analogía de estudiar para un examen memorizando»	Que lo explicáis con vuestras palabras
Interpretar una curva	«Mi pérdida de entrenamiento baja pero la de validación sube a partir de la época 12. ¿Qué significa y qué puedo probar?»	Que probáis lo que sugiere y comprobáis el efecto
Entender un error de Keras	«Me da este error: *[mensaje]* con este código: *[código]*»	Que entendéis la causa
Revisar el informe	«Léelo como un directivo y dime qué no se entiende»	Que no cambie cifras

Ayuda 2 · Errores frecuentes (y cómo evitarlos)

Error	Por qué ocurre	Cómo evitarlo
Escalar con todos los datos	Se olvida la fuga	`fit` del escalador solo con el entrenamiento
Mirar el test para elegir la variante	«Para saber cuál va mejor»	Solo validación; el test, una sola vez
Comparar con una sola semilla	«Ya ha salido mejor»	Varias semillas y media ± desviación
Entrenar 500 épocas	«Cuanto más, mejor»	Máximo 40 épocas y parada temprana
Concluir «la red es mejor» por 0,3 puntos	Se mira solo el número	Comparad los intervalos y añadid el coste
Informe sin recomendación	Se queda en los números	Terminad con «Recomendamos… porque…»
Una celda da error con `...`	Falta rellenar el hueco de un ejercicio	Completad los ejercicios en orden, del 1 al 4
Subir el cuaderno sin resultados	Se descarga sin haberlo ejecutado	*Reiniciar y ejecutar todo* y después descargar el `.ipynb`

Ayuda 3 · Si os atascáis

1. **La red no aprende (la pérdida no baja):** revisad que los datos estén escalados y probad una tasa de aprendizaje menor.
2. **Las curvas son ruidosas:** es normal en datos pequeños; usad varias semillas y promediad.
3. **No entendemos las curvas:** repasad `S10_03_regularizacion`, que tiene un ejemplo comentado, y el código explicado de `S10_04`.
4. Preguntad al equipo y, después, al docente. Más ayuda: Cómo se trabaja un reto.